ChatGPT Get Pro Limited personalization

hello , i want to have functions in Python for this app:
the app is about transport system to make it easier and more convenient for passengers to use buses and trains. the ticket should be for single ride, a set period or in bundles.
Passengers should be able to buy tickets through a mobile application or other digital interface. When a passenger selects a ticket, the system needs to determine how much they should pay based on who they are and the type of ticket they choose. For example, a student may be eligible for a discounted pass, while a senior might get a different offer.
In practice, passengers may purchase multiple tickets at once, and the system must handle different quantities and combinations reliably. There may also be situations where unexpected or unusual inputs occur, such as selecting a ticket without specifying a valid quantity.
The city expects the system to calculate ticket prices correctly and consistently, even during busy times when many passengers are buying tickets at once. The system should also protect passengers' personal and payment information.

Show less

Here are some clean and reusable Python functions you can use as a starting point for your transport ticket system.

The design includes:

- Different passenger types (`adult` , `student` , `senior`)
- Different ticket types (`single` , `period` , `bundle`)
- Discount handling
- Multiple ticket purchases
- Validation of invalid inputs
- Secure handling concepts (without storing sensitive data)
- Thread-safe purchase simulation for busy systems

Python Run

```
from threading import Lock

# ----------------------------
# Ticket Prices
# ----------------------------

BASE_PRICES = {
    "single": 30,
    "period": 800,
    "bundle": 250
}

# ----------------------------
# Passenger Discounts
# ----------------------------

DISCOUNTS = {
    "adult": 0.0,
    "student": 0.25,   # 25% discount
    "senior": 0.40     # 40% discount
}

# ----------------------------
# Thread lock for safe purchases
# ----------------------------

purchase_lock = Lock()

# ----------------------------
# Validate Passenger Type
# ----------------------------

def validate_passenger_type(passenger_type):
    if passenger_type not in DISCOUNTS:
        raise ValueError(f"Invalid passenger type: {passenger_type}")

# ----------------------------
# Validate Ticket Type
# ----------------------------

def validate_ticket_type(ticket_type):
    if ticket_type not in BASE_PRICES:
        raise ValueError(f"Invalid ticket type: {ticket_type}")

# ----------------------------
# Validate Quantity
# ----------------------------

def validate_quantity(quantity):
    if not isinstance(quantity, int):
        raise TypeError("Quantity must be an integer")
```

```
        if quantity <= 0:
            raise ValueError("Quantity must be greater than 0")

    # ----------------------------
    # Calculate Single Ticket Price
    # ----------------------------

    def calculate_ticket_price(passenger_type, ticket_type):
        validate_passenger_type(passenger_type)
        validate_ticket_type(ticket_type)

        base_price = BASE_PRICES[ticket_type]
        discount = DISCOUNTS[passenger_type]

        final_price = base_price * (1 - discount)

        return round(final_price, 2)

    # ----------------------------
    # Calculate Total Purchase
    # ----------------------------

    def calculate_total_purchase(passenger_type, purchases):
        """
        purchases example:
        [
            {"ticket_type": "single", "quantity": 2},
            {"ticket_type": "bundle", "quantity": 1}
        ]
        """

        validate_passenger_type(passenger_type)

        total = 0

        for item in purchases:
            ticket_type = item.get("ticket_type")
            quantity = item.get("quantity")

            validate_ticket_type(ticket_type)
            validate_quantity(quantity)

            ticket_price = calculate_ticket_price(
                passenger_type,
                ticket_type
            )

            total += ticket_price * quantity

        return round(total, 2)

    # ----------------------------
```

```
# Simulate Secure Payment
# ----------------------------

def process_payment(amount, payment_token):
    """
    Simulates payment processing.
    Never store raw card details.
    """

    if not payment_token:
        raise ValueError("Invalid payment token")

    if amount <= 0:
        raise ValueError("Amount must be positive")

    return {
        "status": "success",
        "amount_paid": amount
    }

# ----------------------------
# Thread-safe Ticket Purchase
# ----------------------------

def purchase_tickets(passenger_type, purchases, payment_token):
    with purchase_lock:

        total_price = calculate_total_purchase(
            passenger_type,
            purchases
        )

        payment_result = process_payment(
            total_price,
            payment_token
        )

        return {
            "tickets": purchases,
            "total_price": total_price,
            "payment": payment_result
        }

# ----------------------------
# Example Usage
# ----------------------------

if __name__ == "__main__":

    purchases = [
        {"ticket_type": "single", "quantity": 2},
        {"ticket_type": "period", "quantity": 1}
```

```
    ]

    result = purchase_tickets(
        passenger_type="student",
        purchases=purchases,
        payment_token="secure_token_123"
    )

    print(result)
```

Example output:

Python Run

```
{
    'tickets': [
        {'ticket_type': 'single', 'quantity': 2},
        {'ticket_type': 'period', 'quantity': 1}
    ],
    'total_price': 645.0,
    'payment': {
        'status': 'success',
        'amount_paid': 645.0
    }
}
```

Some good next features you could add:

- QR-code ticket generation
- Expiration time for tickets
- Database integration
- Login/authentication
- Refund handling
- Peak/off-peak pricing
- Family/group discounts
- Real payment gateway integration
- Encryption for passenger data
- Unit tests with `pytest`

ok this is good thanks

You're welcome! Your app idea is actually very solid for practicing:

- Python functions
- Object-oriented programming
- Error handling
- Security concepts
- Concurrent systems
- Real-world software design

As you continue, a good next step would be turning this into classes like:

- `Passenger`
- `Ticket`
- `Purchase`
- `PaymentService`

That will make the system cleaner and easier to expand later into a real mobile/backend application.

You could also later connect it to:

- a frontend (`React` , `Vue` , or mobile app)
- a database (`SQLite` or `PostgreSQL`)
- an API framework like Flask or FastAPI

Good foundation overall.